Module 2: Core Components and Layout

Introduction

In this module, we'll explore Streamlit's core components for displaying text, data, and creating professional layouts. These building blocks form the foundation of every Streamlit application, from simple data viewers to complex dashboards. By the end of this module, you'll be able to create visually appealing and well-structured applications that effectively communicate data insights.

Part 1: Display Text and Markdown

Streamlit provides multiple ways to display text, each serving a specific purpose. Understanding when to use each method will help you create clear and professional applications.

st.title() - Page Titles

The `st.title()` function creates the largest text element, typically used once at the top of your application to establish the main purpose.

```
import streamlit as st

st.title("Data Analysis Dashboard")
# Creates large, bold text that serves as the main application title
```

st.header() and st.subheader() - Section Organization

Headers and subheaders create visual hierarchy and organize content into logical sections.

```
st.header("Sales Performance Overview")
# Creates a section header - second largest text size

st.subheader("Regional Breakdown")
# Creates a subsection - third largest text size
```

st.text() - Plain Text Display

Use `st.text()` when you need to display preformatted text or maintain exact spacing and line breaks.

```
st.text("This is plain text with fixed formatting")
st.text("""Line 1
Line 2
    Indented Line 3""")
# Preserves spacing and formatting exactly as written
```

st.markdown() - Rich Text Formatting

Markdown provides extensive formatting options including bold, italic, links, lists, and more.

```
st.markdown("**Bold text** and *italic text*")
st.markdown("[Click here](https://streamlit.io) for Streamlit documentation")
st.markdown("""
### Features
- Bullet point 1
- Bullet point 2
  - Nested point

1. Numbered item
2. Another item
""")
```

st.write() - The Universal Display Function

`st.write()` is Streamlit's most versatile function. It automatically detects the data type and displays it appropriately.

```
# st.write() can handle multiple data types
st.write("Simple text string")
st.write(123)  # Numbers
st.write({"key": "value"})  # Dictionaries
st.write(["item1", "item2"])  # Lists

import pandas as pd
df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
st.write(df)  # DataFrames

# Can combine multiple items
st.write("The dataframe has", len(df), "rows")
```

st.code() - Code Display with Syntax Highlighting

Display code snippets with proper syntax highlighting for various programming languages.

```
# Python code with syntax highlighting
python_code = """
def calculate_average(numbers):
    return sum(numbers) / len(numbers)

result = calculate_average([1, 2, 3, 4, 5])
print(f"Average: {result}")
"""

st.code(python_code, language="python")

# SQL example
sql_query = """
SELECT customer_name, order_total
FROM orders
WHERE order_date >= '2024-01-01'
ORDER BY order_total DESC
"""

st.code(sql_query, language="sql")
```

st.latex() - Mathematical Expressions

Display mathematical formulas and equations using LaTeX notation.

```
st.latex(r"\sum_{i=1}^{n} x_i = x_1 + x_2 + ... + x_n")
st.latex(r"e^{i\pi} + 1 = 0")  # Euler's identity
st.latex(r"\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}")
```

Part 2: Displaying Data

Streamlit offers specialized components for presenting different types of data effectively.

st.dataframe() vs st.table() - Tabular Data Display

Understanding the difference between these two functions is crucial for choosing the right display method.

st.dataframe() - Interactive Tables

```
import pandas as pd
import numpy as np

# Create sample data
df = pd.DataFrame(
    np.random.randn(50, 5),
    columns=["A", "B", "C", "D", "E"]
)

# Interactive dataframe with sorting and scrolling
st.dataframe(df)

# Customize display
st.dataframe(
    df,
    height=300,  # Set fixed height
    width='stretch'  # Use full width, width='content' for content width
)

# Highlight specific values
st.dataframe(
    df.style.highlight_max(axis=0, color="lightgreen")
)
```

st.table() - Static Tables

```
# Static table for small, important data
summary_data = pd.DataFrame({
    "Metric": ["Revenue", "Profit", "Growth"],
    "Value": ["$1.2M", "$300K", "15%"]
})

st.table(summary_data)
# Creates a static table without scrolling or interaction
```

st.metric() - Key Performance Indicators

Display important metrics with optional delta indicators for tracking changes.

```
# Simple metric
st.metric(label="Total Sales", value="$45,230")

# Metric with delta (positive)
st.metric(
    label="Revenue",
    value="$12,500",
    delta="$1,250",
    delta_color="normal"  # Green for positive
)

# Metric with negative delta
st.metric(
    label="Expenses",
    value="$8,300",
    delta="-$500",
    delta_color="inverse"  # Green for negative (good for expenses)
)

# Multiple metrics in columns
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Users", "1,234", "+12%")
with col2:
    st.metric("Revenue", "$45.2K", "+5.1%")
with col3:
    st.metric("Efficiency", "87%", "-2%", delta_color="normal")
```

st.json() - JSON Data Display

Display JSON data with proper formatting and syntax highlighting.

```
import json

# Sample JSON data
api_response = {
    "status": "success",
    "data": {
        "user_id": 123,
        "username": "john_doe",
        "email": "john@example.com",
        "preferences": {
            "theme": "dark",
```

```
            "notifications": True,
            "language": "en"
        },
        "tags": ["premium", "verified", "active"]
    },
    "timestamp": "2024-11-15T10:30:00Z"
}

st.json(api_response)
# Displays with collapsible sections and syntax highlighting
```

Part 3: Layout and Container Management

Creating well-organized layouts is essential for building professional applications. Streamlit provides several container types for organizing content.

st.columns() - Side-by-Side Layout

Columns allow you to place elements horizontally across the page.

```
# Create equal-width columns
col1, col2, col3 = st.columns(3)

with col1:
    st.header("Column 1")
    st.write("Content for the first column")
    st.button("Button 1")

with col2:
    st.header("Column 2")
    st.write("Content for the second column")
    st.selectbox("Options", ["A", "B", "C"])

with col3:
    st.header("Column 3")
    st.write("Content for the third column")
    st.checkbox("Enable feature")

# Columns with different widths
col1, col2 = st.columns([2, 1])  # First column twice as wide

with col1:
    st.write("Wide column for main content")
```

```
with col2:
    st.write("Narrow column for sidebar info")

# Using columns with gap parameter
col1, col2 = st.columns(2, gap="large")

with col1:
    st.write("This is column 1")

with col2:
    st.write("This is column 2")
```

st.tabs() - Tabbed Interface

Tabs organize related content into switchable sections.

```
tab1, tab2, tab3 = st.tabs(["Data Entry", "Visualization", "Settings"])

with tab1:
    st.header("Enter Your Data")
    name = st.text_input("Name")
    age = st.number_input("Age", min_value=0, max_value=120)
    st.write(f"Hello {name}, you are {age} years old")

with tab2:
    st.header("Data Visualization")
    import pandas as pd
    import numpy as np

    chart_data = pd.DataFrame(
        np.random.randn(20, 3),
        columns=["Series 1", "Series 2", "Series 3"]
    )
    st.line_chart(chart_data)

with tab3:
    st.header("Application Settings")
    dark_mode = st.checkbox("Enable dark mode")
    notifications = st.checkbox("Enable notifications")
    language = st.selectbox("Language", ["English", "Spanish", "French"])
```

st.expander() - Collapsible Sections

Expanders help manage screen space by hiding detailed information until needed.

```
# Basic expander
with st.expander("Click to see more details"):
    st.write("This content is hidden by default")
    st.write("Users can click to expand and see this information")

    # Can include any Streamlit elements
    st.selectbox("Hidden option", ["Option 1", "Option 2"])
    st.slider("Hidden slider", 0, 100, 50)

# Expander that starts expanded
with st.expander("Important Information", expanded=True):
    st.warning("This section is expanded by default")
    st.write("Contains critical information users should see")

# Multiple expanders for FAQ section
st.header("Frequently Asked Questions")

with st.expander("What is Streamlit?"):
    st.write("Streamlit is a Python framework for creating data applications.")

with st.expander("How do I install Streamlit?"):
    st.code("pip install streamlit", language="bash")

with st.expander("Can I deploy Streamlit apps?"):
    st.write("Yes, you can deploy to Streamlit Community Cloud, AWS, GCP, and more.")
```

st.container() - Logical Grouping

Containers group elements together and allow for out-of-order insertion.

```
# Basic container
with st.container():
    st.write("This is inside a container")
    st.button("Container button")

# Container with border
with st.container(border=True):
    st.write("This container has a visible border")
    st.metric("Metric inside container", "123")
```

```
# Out-of-order insertion
container = st.container()
st.write("This appears below the container")

# Add content to container later
with container:
    st.write("This appears above even though defined later")

# Nested containers
with st.container():
    st.write("Outer container")
    with st.container(border=True):
        st.write("Inner container with border")
```

st.sidebar - Navigation and Controls

The sidebar provides a collapsible panel for navigation and controls.

```
# Add elements to sidebar
with st.sidebar:
    st.title("Navigation")

    # Page selection
    page = st.selectbox("Choose a page", ["Home", "Analysis", "Settings"])

    st.divider()

    # Filters
    st.header("Filters")
    date_range = st.date_input("Date range", [])
    category = st.multiselect("Categories", ["A", "B", "C", "D"])

    st.divider()

    # Settings
    st.header("Settings")
    show_raw_data = st.checkbox("Show raw data")
    enable_cache = st.checkbox("Enable caching", value=True)

# Main content area responds to sidebar selections
if page == "Home":
    st.title("Welcome to the Dashboard")
elif page == "Analysis":
```

```
    st.title("Data Analysis")
    if show_raw_data:
        st.write("Raw data would be displayed here")
```

Hands-on Practice

Practice 1: Build a Dashboard Combining Text, Tables, and Metrics

Create a comprehensive sales dashboard that demonstrates all text and data display components.

```
import streamlit as st
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Page configuration
st.set_page_config(
    page_title="Sales Dashboard",
    layout="wide"
)

# Main title
st.title("Sales Performance Dashboard")
st.markdown("---")

# Generate sample data
np.random.seed(42)
dates = pd.date_range(start="2024-01-01", end="2024-10-31", freq="D")
sales_data = pd.DataFrame({
    "Date": dates,
    "Sales": np.random.randint(1000, 5000, len(dates)),
    "Orders": np.random.randint(20, 100, len(dates)),
    "Region": np.random.choice(["North", "South", "East", "West"], len(dates)),
    "Product": np.random.choice(["Product A", "Product B", "Product C"], len(dates))
})

# Calculate metrics
total_sales = sales_data["Sales"].sum()
total_orders = sales_data["Orders"].sum()
avg_order_value = total_sales / total_orders
yesterday_sales = sales_data.iloc[-2]["Sales"]
today_sales = sales_data.iloc[-1]["Sales"]
```

```
sales_change = ((today_sales - yesterday_sales) / yesterday_sales) * 100

# Display metrics in columns
st.header("Key Performance Indicators")
col1, col2, col3, col4 = st.columns(4)

with col1:
    st.metric(
        label="Total Sales",
        value=f"${total_sales:,.0f}",
        delta=f"{sales_change:.1f}% from yesterday"
    )

with col2:
    st.metric(
        label="Total Orders",
        value=f"{total_orders:,}",
        delta="↑ 12 orders"
    )

with col3:
    st.metric(
        label="Average Order Value",
        value=f"${avg_order_value:.2f}",
        delta="-$2.50",
        delta_color="inverse"
    )

with col4:
    st.metric(
        label="Conversion Rate",
        value="3.2%",
        delta="+0.3%"
    )

st.markdown("---")

# Sales by region and product
st.header("Sales Analysis")

col1, col2 = st.columns(2)

with col1:
    st.subheader("Sales by Region")
```

```
    region_sales = sales_data.groupby("Region")["Sales"].sum().reset_index()
    region_sales = region_sales.sort_values("Sales", ascending=False)
    st.dataframe(
        region_sales.style.format({"Sales": "${:,.0f}"}),
        use_container_width=True,
        hide_index=True
    )

with col2:
    st.subheader("Sales by Product")
    product_sales = sales_data.groupby("Product")["Sales"].sum().reset_index()
    product_sales["Percentage"] = (product_sales["Sales"] / product_sales["Sales"].sum() * 100)
    st.table(
        product_sales.style.format({
            "Sales": "${:,.0f}",
            "Percentage": "{:.1f}%"
        })
    )

st.markdown("---")

# Recent transactions
st.header("Recent Transactions")
st.markdown("*Showing last 10 transactions*")

recent_transactions = sales_data.tail(10)[["Date", "Product", "Region", "Sales", "Orders"]]
recent_transactions["Date"] = recent_transactions["Date"].dt.strftime("%Y-%m-%d")

st.dataframe(
    recent_transactions.style.format({"Sales": "${:,.0f}"}),
    use_container_width=True,
    hide_index=True
)

# Display code snippet
st.markdown("---")
st.subheader("Sample Analysis Code")
code = """
# Calculate total sales by region
region_totals = sales_data.groupby('Region')['Sales'].sum()
top_region = region_totals.idxmax()
print(f"Top performing region: {top_region}")
"""
st.code(code, language="python")
```

```
# Mathematical formula
st.subheader("Average Order Value Formula")
st.latex(r"AOV = \frac{\sum_{i=1}^{n} Sales_i}{\sum_{i=1}^{n} Orders_i}")

# JSON configuration display
st.subheader("Dashboard Configuration")
config = {
    "dashboard": {
        "title": "Sales Dashboard",
        "refresh_rate": "5 minutes",
        "filters": {
            "date_range": "Last 30 days",
            "regions": ["North", "South", "East", "West"],
            "products": ["Product A", "Product B", "Product C"]
        }
    }
}
st.json(config)
```

Practice 2: Create Multi-Column and Tabbed Layouts

Build an employee management interface with a sophisticated layout.

```
import streamlit as st
import pandas as pd
import numpy as np

st.set_page_config(page_title="Employee Management System", layout="wide")

st.title("Employee Management System")
st.markdown("Comprehensive employee data management interface")

# Create sample employee data
np.random.seed(42)
employees = pd.DataFrame({
    "ID": range(1001, 1021),
    "Name": [f"Employee {i}" for i in range(1, 21)],
    "Department": np.random.choice(["Sales", "IT", "HR", "Finance"], 20),
    "Salary": np.random.randint(40000, 120000, 20),
    "Performance": np.random.choice(["Excellent", "Good", "Average", "Needs Improvement"],
20),
    "Years": np.random.randint(1, 15, 20)
```

```
})

# Main tabs
tab1, tab2, tab3, tab4 = st.tabs(["Overview", "Employee Details", "Department Analysis",
"Reports"])

with tab1:
    st.header("Company Overview")

    # Metrics row
    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric("Total Employees", len(employees))
    with col2:
        st.metric("Average Salary", f"${employees['Salary'].mean():,.0f}")
    with col3:
        st.metric("Departments", employees["Department"].nunique())
    with col4:
        st.metric("Avg. Tenure", f"{employees['Years'].mean():.1f} years")

    st.divider()

    # Two column layout for charts
    col1, col2 = st.columns(2)

    with col1:
        st.subheader("Employees by Department")
        dept_counts = employees["Department"].value_counts()
        st.bar_chart(dept_counts)

    with col2:
        st.subheader("Salary Distribution")

        # Fix: Create salary bins with string labels
        # Method 1: Using custom labels
        salary_bins = pd.cut(employees["Salary"], bins=5)

        # Convert intervals to strings for display
        salary_dist = salary_bins.value_counts().sort_index()

        # Create a DataFrame with string labels
        salary_df = pd.DataFrame({
            'Salary Range': [str(interval) for interval in salary_dist.index],
```

```
        'Count': salary_dist.values
    })

    # Use st.bar_chart with the index as x-axis
    st.bar_chart(salary_df.set_index('Salary Range'))

    # Alternative Method: You could also use specific bin labels
    # bins = [40000, 55000, 70000, 85000, 100000, 120000]
    # labels = ['40K-55K', '55K-70K', '70K-85K', '85K-100K', '100K-120K']
    # salary_bins = pd.cut(employees["Salary"], bins=bins, labels=labels)
    # salary_dist = salary_bins.value_counts().sort_index()
    # st.bar_chart(salary_dist)

with tab2:
    st.header("Employee Information")

    # Three column layout for filters
    col1, col2, col3 = st.columns(3)

    with col1:
        dept_filter = st.multiselect(
            "Filter by Department",
            employees["Department"].unique(),
            default=employees["Department"].unique()
        )

    with col2:
        perf_filter = st.multiselect(
            "Filter by Performance",
            employees["Performance"].unique(),
            default=employees["Performance"].unique()
        )

    with col3:
        salary_range = st.slider(
            "Salary Range",
            int(employees["Salary"].min()),
            int(employees["Salary"].max()),
            (int(employees["Salary"].min()), int(employees["Salary"].max()))
        )

    # Apply filters
    filtered_employees = employees[
        (employees["Department"].isin(dept_filter)) &
```

```
        (employees["Performance"].isin(perf_filter)) &
        (employees["Salary"].between(salary_range[0], salary_range[1]))
    ]

    # Display filtered data
    st.subheader(f"Showing {len(filtered_employees)} employees")
    st.dataframe(
        filtered_employees.style.format({"Salary": "${:,.0f}"}),
        use_container_width=True
    )

with tab3:
    st.header("Department Analysis")

    # Create department summary
    dept_summary = employees.groupby("Department").agg({
        "ID": "count",
        "Salary": ["mean", "min", "max"],
        "Years": "mean"
    }).round(0)

    dept_summary.columns = ["Employees", "Avg Salary", "Min Salary", "Max Salary", "Avg
Tenure"]

    # Two column layout
    col1, col2 = st.columns([2, 1])

    with col1:
        st.subheader("Department Statistics")
        st.table(dept_summary.style.format({
            "Avg Salary": "${:,.0f}",
            "Min Salary": "${:,.0f}",
            "Max Salary": "${:,.0f}",
            "Avg Tenure": "{:.1f} years"
        }))

    with col2:
        st.subheader("Quick Stats")
        selected_dept = st.selectbox("Select Department", employees["Department"].unique())
        dept_data = employees[employees["Department"] == selected_dept]

        st.metric("Team Size", len(dept_data))
        st.metric("Budget", f"${dept_data['Salary'].sum():,.0f}")
```

```
        # Fix: Handle case where there might be no "Excellent" performers
        excellent_performers = dept_data[dept_data["Performance"] == "Excellent"]["Name"]
        if len(excellent_performers) > 0:
            top_performer = excellent_performers.iloc[0]
        else:
            top_performer = "N/A"
        st.metric("Top Performer", top_performer)

with tab4:
    st.header("Generate Reports")

    col1, col2 = st.columns([1, 2])

    with col1:
        st.subheader("Report Options")
        report_type = st.radio(
            "Select Report Type",
            ["Performance Summary", "Salary Report", "Tenure Analysis"]
        )

        include_charts = st.checkbox("Include visualizations", value=True)
        include_raw_data = st.checkbox("Include raw data")

        if st.button("Generate Report", type="primary"):
            with col2:
                st.subheader(f"Generated {report_type}")
                st.info(f"Report generated at {pd.Timestamp.now().strftime('%Y-%m-%d
%H:%M:%S')}")

                if report_type == "Performance Summary":
                    perf_summary = employees["Performance"].value_counts()
                    st.write("Performance distribution across company:")
                    if include_charts:
                        st.bar_chart(perf_summary)
                    if include_raw_data:
                        st.dataframe(perf_summary)

                elif report_type == "Salary Report":
                    st.write(f"Total Payroll: ${employees['Salary'].sum():,.0f}")
                    st.write(f"Average Salary: ${employees['Salary'].mean():,.0f}")
                    st.write(f"Median Salary: ${employees['Salary'].median():,.0f}")
                    if include_charts:
                        st.line_chart(employees.groupby("Department")["Salary"].mean())
```

```
        elif report_type == "Tenure Analysis":
            st.write(f"Average Tenure: {employees['Years'].mean():.1f} years")
            st.write(f"Longest Tenure: {employees['Years'].max()} years")
            if include_charts:
                st.bar_chart(employees["Years"].value_counts().sort_index())
```

Practice 3: Add Collapsible Sections and Sidebar Filters

Create a data exploration tool with expandable sections and comprehensive filtering.

```
import streamlit as st
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

st.set_page_config(
    page_title="Data Explorer",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Generate sample dataset
np.random.seed(42)
n_records = 1000

data = pd.DataFrame({
    "ID": range(1, n_records + 1),
    "Date": pd.date_range(start="2024-01-01", periods=n_records, freq="H"),
    "Category": np.random.choice(["Electronics", "Clothing", "Food", "Books", "Home"],
n_records),
    "Subcategory": np.random.choice(["Type A", "Type B", "Type C"], n_records),
    "Value": np.random.exponential(100, n_records),
    "Quantity": np.random.poisson(5, n_records),
    "Status": np.random.choice(["Active", "Pending", "Completed", "Cancelled"], n_records),
    "Priority": np.random.choice(["Low", "Medium", "High", "Critical"], n_records),
    "Score": np.random.uniform(0, 100, n_records)
})

# Sidebar configuration
with st.sidebar:
    st.title("Data Explorer Settings")

    # Data Filters Section
```

```
st.header("Filters")

with st.expander("Date Range", expanded=True):
    date_range = st.date_input(
        "Select date range",
        value=(data["Date"].min(), data["Date"].max()),
        min_value=data["Date"].min(),
        max_value=data["Date"].max()
    )

with st.expander("Categories", expanded=True):
    selected_categories = st.multiselect(
        "Select categories",
        data["Category"].unique(),
        default=data["Category"].unique()
    )

    selected_subcategories = st.multiselect(
        "Select subcategories",
        data["Subcategory"].unique(),
        default=data["Subcategory"].unique()
    )

with st.expander("Status & Priority"):
    selected_status = st.multiselect(
        "Status",
        data["Status"].unique(),
        default=data["Status"].unique()
    )

    selected_priority = st.multiselect(
        "Priority",
        data["Priority"].unique(),
        default=["High", "Critical"]
    )

with st.expander("Value Ranges"):
    value_range = st.slider(
        "Value range",
        float(data["Value"].min()),
        float(data["Value"].max()),
        (float(data["Value"].min()), float(data["Value"].max()))
    )
```

```
        score_range = st.slider(
            "Score range",
            0.0,
            100.0,
            (0.0, 100.0)
        )

    st.divider()

    # Display Options
    st.header("Display Options")

    with st.expander("View Settings"):
        show_raw_data = st.checkbox("Show raw data", value=False)
        show_statistics = st.checkbox("Show statistics", value=True)
        show_charts = st.checkbox("Show visualizations", value=True)
        rows_to_display = st.number_input("Rows to display", min_value=10, max_value=1000,
value=100)

    with st.expander("Export Options"):
        export_format = st.radio("Export format", ["CSV", "Excel", "JSON"])
        if st.button("Export Data"):
            # Simulate export functionality
            if export_format == "CSV":
                csv = filtered_data.to_csv(index=False)
                st.download_button(
                    label="Download CSV",
                    data=csv,
                    file_name=f"data_export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv",
                    mime="text/csv"
                )
            elif export_format == "JSON":
                json_str = filtered_data.to_json(orient='records', indent=2)
                st.download_button(
                    label="Download JSON",
                    data=json_str,
                    file_name=f"data_export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json",
                    mime="application/json"
                )
            st.success(f"Data ready for download as {export_format}")

# Apply filters
filtered_data = data[
    (data["Date"] >= pd.Timestamp(date_range[0])) &
```

```
    (data["Date"] <= pd.Timestamp(date_range[1])) &
    (data["Category"].isin(selected_categories)) &
    (data["Subcategory"].isin(selected_subcategories)) &
    (data["Status"].isin(selected_status)) &
    (data["Priority"].isin(selected_priority)) &
    (data["Value"].between(value_range[0], value_range[1])) &
    (data["Score"].between(score_range[0], score_range[1]))
]

# Main content area
st.title("Data Analysis Platform")
st.markdown(f"Analyzing {len(filtered_data)} of {len(data)} records based on current filters")

# Summary metrics
with st.container():
    col1, col2, col3, col4, col5 = st.columns(5)

    with col1:
        st.metric("Total Records", f"{len(filtered_data):,}")
    with col2:
        st.metric("Total Value", f"${filtered_data['Value'].sum():,.0f}")
    with col3:
        st.metric("Avg Score", f"{filtered_data['Score'].mean():.1f}")
    with col4:
        st.metric("Total Quantity", f"{filtered_data['Quantity'].sum():,}")
    with col5:
        completion_rate = (filtered_data["Status"] == "Completed").mean() * 100
        st.metric("Completion Rate", f"{completion_rate:.1f}%")

st.divider()

# Main content with expanders
col1, col2 = st.columns([2, 1])

with col1:
    # Category Analysis
    with st.expander("Category Analysis", expanded=show_statistics):
        st.subheader("Performance by Category")

        category_summary = filtered_data.groupby("Category").agg({
            "Value": ["sum", "mean", "count"],
            "Score": "mean",
            "Quantity": "sum"
        }).round(2)
```

```
        category_summary.columns = ["Total Value", "Avg Value", "Count", "Avg Score", "Total
Qty"]
        st.dataframe(
            category_summary.style.format({
                "Total Value": "${:,.0f}",
                "Avg Value": "${:,.2f}",
                "Avg Score": "{:.1f}",
                "Total Qty": "{:,.0f}"
            }),
            use_container_width=True
        )

    # Time Series Analysis
    with st.expander("Time Series Analysis", expanded=show_charts):
        st.subheader("Trends Over Time")

        # Aggregate by date
        daily_data = filtered_data.groupby(filtered_data["Date"].dt.date)["Value"].sum()

        tab1, tab2 = st.tabs(["Daily Totals", "Cumulative"])

        with tab1:
            st.line_chart(daily_data)

        with tab2:
            st.line_chart(daily_data.cumsum())

    # Status Distribution
    with st.expander("Status & Priority Distribution"):
        subcol1, subcol2 = st.columns(2)

        with subcol1:
            st.subheader("Status Breakdown")
            status_counts = filtered_data["Status"].value_counts()
            st.bar_chart(status_counts)

        with subcol2:
            st.subheader("Priority Distribution")
            priority_counts = filtered_data["Priority"].value_counts()
            st.bar_chart(priority_counts)

with col2:
    # Statistics Panel
```

```
    with st.expander("Quick Statistics", expanded=True):
        st.subheader("Statistical Summary")

        stats = pd.DataFrame({
            "Metric": ["Mean Value", "Median Value", "Std Dev", "Min Value", "Max Value"],
            "Value": [
                f"${filtered_data['Value'].mean():.2f}",
                f"${filtered_data['Value'].median():.2f}",
                f"${filtered_data['Value'].std():.2f}",
                f"${filtered_data['Value'].min():.2f}",
                f"${filtered_data['Value'].max():.2f}"
            ]
        })
        st.table(stats)

    # Top Performers
    with st.expander("Top Performers"):
        st.subheader("Top 5 by Value")
        top_5 = filtered_data.nlargest(5, "Value")[["ID", "Category", "Value", "Score"]]
        st.table(top_5.style.format({"Value": "${:,.2f}", "Score": "{:.1f}"}))

# Raw Data Display
if show_raw_data:
    with st.expander("Raw Data View", expanded=False):
        st.subheader(f"Showing first {rows_to_display} rows")

        display_cols = st.multiselect(
            "Select columns to display",
            data.columns.tolist(),
            default=data.columns.tolist()
        )

        st.dataframe(
            filtered_data[display_cols].head(rows_to_display),
            use_container_width=True
        )

# Advanced Analysis Section
with st.expander("Advanced Analysis Tools"):
    analysis_type = st.selectbox(
        "Select analysis type",
        ["Correlation Matrix", "Pivot Table", "Statistical Tests", "Heatmap View"]
    )
```

```
if analysis_type == "Correlation Matrix":
    numeric_cols = filtered_data.select_dtypes(include=[np.number]).columns
    corr_matrix = filtered_data[numeric_cols].corr()
    st.write("Correlation between numeric variables:")

    # Custom color coding function without matplotlib
    def highlight_correlation(val):
        """Apply color based on correlation strength"""
        if pd.isna(val):
            return ''

        abs_val = abs(val)
        if val < -0.5:
            color = 'background-color: #ff6b6b; color: white'  # Strong negative
        elif val < 0:
            color = 'background-color: #ffd3d3'  # Weak negative
        elif val < 0.5:
            color = 'background-color: #d3ffd3'  # Weak positive
        elif val < 1:
            color = 'background-color: #6bff6b; color: white'  # Strong positive
        else:
            color = 'background-color: #4CAF50; color: white'  # Perfect correlation (1)

        return color

    # Apply styling and display
    styled_corr = corr_matrix.style.format("{:.2f}").applymap(highlight_correlation)
    st.dataframe(styled_corr, use_container_width=True)

    # Add legend for correlation colors
    st.caption("""
    **Color Legend:**
    - 🔴 Dark Red: Strong negative correlation (< -0.5)
    - 🌸 Light Red: Weak negative correlation (0 to -0.5)
    - 🌱 Light Green: Weak positive correlation (0 to 0.5)
    - 🟢 Dark Green: Strong positive correlation (> 0.5)
    """)

elif analysis_type == "Pivot Table":
    pivot_table = pd.pivot_table(
        filtered_data,
        values="Value",
        index="Category",
        columns="Status",
```

```
        aggfunc="sum",
        fill_value=0
    )
    st.write("Value by Category and Status:")

    # Add row and column totals
    pivot_table['Total'] = pivot_table.sum(axis=1)
    pivot_table.loc['Total'] = pivot_table.sum()

    st.dataframe(
        pivot_table.style.format("${:,.0f}").highlight_max(axis=0, color='lightgreen'),
        use_container_width=True
    )

elif analysis_type == "Statistical Tests":
    st.write("Comprehensive Statistical Analysis for Value Column:")

    # Calculate all statistics
    value_data = filtered_data["Value"]

    # Basic statistics
    col1, col2 = st.columns(2)

    with col1:
        st.write("**Central Tendency:**")
        st.write(f"• Mean: ${value_data.mean():.2f}")
        st.write(f"• Median: ${value_data.median():.2f}")
        mode_values = value_data.mode()
        mode_val = mode_values.iloc[0] if len(mode_values) > 0 else "N/A"
        st.write(f"• Mode: ${mode_val:.2f}" if mode_val != "N/A" else "• Mode: N/A")

        st.write("**Dispersion:**")
        st.write(f"• Std Dev: ${value_data.std():.2f}")
        st.write(f"• Variance: ${value_data.var():.2f}")
        st.write(f"• Range: ${value_data.max() - value_data.min():.2f}")

    with col2:
        st.write("**Distribution Shape:**")
        st.write(f"• Skewness: {value_data.skew():.3f}")
        st.write(f"• Kurtosis: {value_data.kurtosis():.3f}")

        st.write("**Percentiles:**")
        st.write(f"• 25th: ${value_data.quantile(0.25):.2f}")
        st.write(f"• 50th: ${value_data.quantile(0.50):.2f}")
```

```
        st.write(f"• 75th: ${value_data.quantile(0.75):.2f}")
        st.write(f"• IQR: ${value_data.quantile(0.75) - value_data.quantile(0.25):.2f}")

elif analysis_type == "Heatmap View":
    st.write("Category vs Status Heatmap (Average Values):")

    # Create heatmap data
    heatmap_data = pd.pivot_table(
        filtered_data,
        values="Value",
        index="Category",
        columns="Priority",
        aggfunc="mean",
        fill_value=0
    )

    # Apply color gradient manually
    def color_scale(val, vmin, vmax):
        """Create color based on value"""
        if pd.isna(val) or vmax == vmin:
            return ''

        # Normalize value between 0 and 1
        norm_val = (val - vmin) / (vmax - vmin)

        # Create gradient from blue (low) to red (high)
        if norm_val < 0.33:
            color = f'background-color: rgba(66, 134, 244, {0.3 + norm_val})'
        elif norm_val < 0.67:
            color = f'background-color: rgba(255, 193, 7, {0.3 + norm_val})'
        else:
            color = f'background-color: rgba(244, 67, 54, {0.3 + norm_val})'

        return color

    # Apply styling
    vmin = heatmap_data.min().min()
    vmax = heatmap_data.max().max()
    styled_heatmap = heatmap_data.style.format("${:.0f}").applymap(
        lambda x: color_scale(x, vmin, vmax)
    )

    st.dataframe(styled_heatmap, use_container_width=True)
    st.caption("Darker colors indicate higher average values")
```

```
# Footer
st.divider()
st.markdown("---")
st.caption(f"Data Explorer v2.0 | Last updated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
```

Key Concepts Summary

Text Display Components

- Use `st.title()` for main application titles
- Use `st.header()` and `st.subheader()` to organize content hierarchically
- Use `st.write()` as a versatile display function for any data type
- Use `st.markdown()` for rich text formatting with links and styling
- Use `st.code()` to display code with syntax highlighting
- Use `st.latex()` for mathematical formulas

Data Display Components

- Use `st.dataframe()` for large, interactive datasets with sorting and searching
- Use `st.table()` for small, static data that should always be visible
- Use `st.metric()` to highlight KPIs with optional delta indicators
- Use `st.json()` to display structured JSON data with collapsible sections

Layout Management

- Use `st.columns()` to create side-by-side layouts with custom width ratios
- Use `st.tabs()` to organize related content into switchable sections
- Use `st.expander()` to create collapsible sections that save screen space
- Use `st.container()` to group elements and enable out-of-order insertion
- Use `st.sidebar` for navigation, filters, and settings that persist across pages

Best Practices

1. **Hierarchy and Organization**
    - Use consistent heading levels to create clear content hierarchy
    - Group related elements using containers or expanders
    - Place filters and controls in the sidebar to maximize content space

2. **Performance Considerations**
    - Use `st.dataframe()` for large datasets (>100 rows)
    - Use `st.table()` for small, important data summaries
    - Implement filters to reduce the amount of displayed data
3. **User Experience**
    - Start expanders collapsed unless content is essential
    - Use tabs to reduce cognitive load and organize complex interfaces
    - Provide clear labels and descriptions for all interactive elements
    - Use metrics to highlight the most important information
4. **Responsive Design**
    - Test layouts with different screen sizes
    - Use `use_container_width=True` for dataframes to adapt to available space
    - Consider mobile users when designing column layouts

Common Pitfalls to Avoid

1. **Overusing Columns**: Don't create more than 4-5 columns as they become too narrow
2. **Nested Expanders**: Avoid deeply nested expanders as they confuse users
3. **Mixing Static and Interactive**: Be consistent - don't mix st.table() and st.dataframe() without reason
4. **Sidebar Overload**: Keep sidebar controls focused and organized
5. **Forgetting Container Borders**: Use border=True to visually separate container content when needed

Next Steps

You now have a solid foundation in Streamlit's core components and layout system. In Module 3, you'll learn about user input widgets and how to create interactive forms and controls that respond to user actions.

Practice creating different layouts and experiment with combining components to build increasingly sophisticated interfaces. Remember that good layout design enhances user experience and makes your data more accessible and understandable.